

Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Структурная и функциональная организация вычислительных
машин

Отчет по лабораторной работе №8

«Организация кэш-памяти»

Вариант 12

Выполнил: студент группы 250541

Семененя К.Н.

Проверил: кандидат технических наук,

Воронов А.А.

Минск 2026

СОДЕРЖАНИЕ

1 Задание	3
2 Блок строки кэш-памяти.....	4
2.1 Расчет параметров кэш-памяти.....	4
2.2 Проектирование блока строки кэш-памяти.....	5
2.3 Моделирование блока строки кэш-памяти.....	7
3 Блок контроллера управления 2-канальным набором кэш-памяти	8
3.1 Проектирование блока контроллера управления 2-канальным набором кэш-памяти.....	8
3.2 Моделирование блока контроллера управления 2-канальным набором кэш-памяти.....	9
4 Блок 2-канального ассоциативного набора кэш-памяти.....	10
4.1 Проектирование блока 2-канального ассоциативного набора кэш-памяти	10
4.2 Моделирование блока 2-канального ассоциативного набора кэш-памяти	11
5 Схема кэш-памяти.....	12
5.1 Проектирование схемы кэш-памяти	12
5.2 Моделирование схемы кэш-памяти	13
3Аключение	14
Приложение А (обязательное) Функциональное моделирование кэш-памяти	15

1 Задание

Лабораторную работу рекомендуется выполнять в новом проекте, не используя предыдущих наработок.

Задача состоит в создании кэш-памяти заданного типа, размера для обслуживания основной памяти с заданным типом замещения. Конкретные характеристики разрабатываемой кэш-памяти приведены в таблице задания 1. Размер шины адреса вычисляется исходя из варианта задания – по нему определяется размер кэш-памяти в байтах, затем определяется размер ОЗУ исходя из кратности его размера объёму кэш-памяти. Адресация ОЗУ по умолчанию – побайтная.

Разрешается использовать функциональное моделирование. В качестве элементов памяти для строк кэш-памяти удобно использовать регистры (например, параметризованный модуль LPM_DFF, в котором легко задать необходимую разрядность).

Содержимое кэш-памяти следует просматривать непосредственно на элементах памяти без вывода на выходные пины, поскольку использование большого количества пинов не позволит Quartus найти подходящую ПЛИС, на которой можно будет реализовать проект.

Таблица 1.1 – Задание

Тип кэш-памяти	Количество строк кэш	Количество слов в блоке данных	Количество модулей, k	Принцип замещения строк в кэш	Кратность размера ОЗУ объёму кэш-памяти	Разрядность шины данных, бит
k-мерное частичное ассоциативное отображение	8	2	2	Наиболее давнего хранения	12	8

2 Блок строки кэш-памяти

2.1 Расчет параметров кэш-памяти

На основании задания для варианта №12 определены следующие исходные данные:

- количество строк кэш-памяти (L): 8;
- количество слов в строке (n): 2;
- разрядность данных (w): 8 бит;
- количество модулей (k): 2;
- кратность объема ОЗУ к объему кэш-памяти (K): 12.

Объем кэш-памяти (V_{cache}) вычисляется по формуле 2.1:

$$V_{cache} = L \cdot n \cdot \frac{w}{8}; \quad (2.1)$$

$$V_{cache} = 8 \cdot 2 \cdot \frac{8}{8} = 16 \text{ байт.}$$

Объем оперативной памяти V_{RAM} вычисляется по формуле 2.2:

$$V_{RAM} = V_{cache} \cdot K; \quad (2.2)$$

$$V_{RAM} = 16 \cdot 12 = 192 \text{ байт.}$$

Ширина шины адреса (N_{addr}) определяется вычисляется по формуле 2.3:

$$N_{addr} = \log_2 V_{RAM}; \quad (2.3)$$

$$N_{addr} = \log_2 192 \approx 8 \text{ бит.}$$

Таким образом, для адресации используется 8-битная шина.

Согласно принципам построения k -мерного частичного ассоциативного отображения (для $k=2$), адресное слово разбивается на три функциональных поля:

- поле смещения N_{offset} (определяет конкретное слово внутри строки кэша);
- поле индекса набора N_{set} (определяет номер набора, в котором может находиться блок данных);
- поле тега (содержит идентификационные биты блока памяти).

Поле смещения определяется по формуле 2.4:

$$N_{offset} = \log_2 n; \quad (2.4)$$

$$N_{offset} = \log_2 2 = 1 \text{ бит.}$$

Поле смещения определяется по формуле 2.5:

$$N_{set} = \log_2 \frac{L}{k}; \quad (2.5)$$

$$N_{set} = \log_2 4 = 2 \text{ бита.}$$

Поле тега определяется по формуле 2.6:

$$N_{tag} = N_{addr} - N_{offset} - N_{set}; \quad (2.6)$$

$$N_{tag} = 8 - 1 - 2 = 5 \text{ бит.}$$

Итоговое распределение бит в адресной шине отображено в таблице 2.1.

Таблица 2.1 – Итоговое распределение бит

Ширина шины адреса, бит	Ширина поля смещения, бит	Ширина поля индекса набора, бит	Ширина поля тега, бит
8	1	2	5

2.2 Проектирование блока строки кэш-памяти

Структурная схема блока строки кэш-памяти отображена на рисунке 2.1.

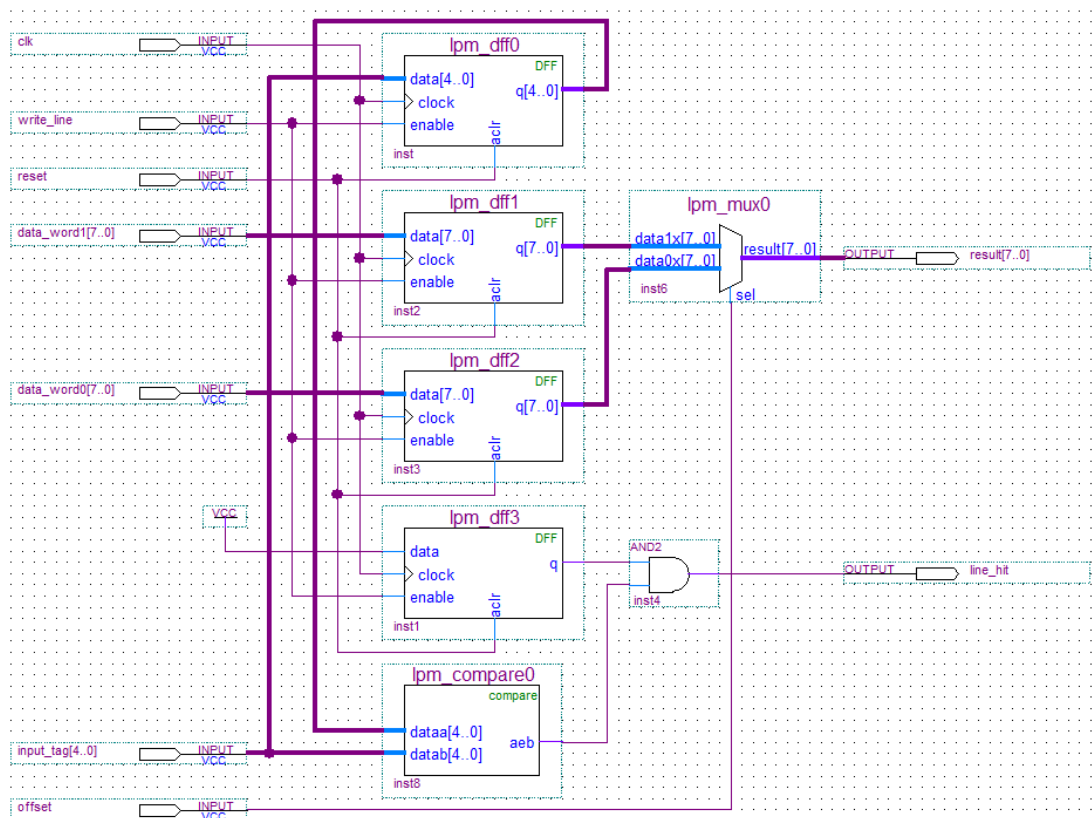


Рисунок 2.1 – Структурная схема блока строки кэш-памяти

Представленная структурная схема блока строки кэш-памяти является базовым модулем хранения – строкой кэш-памяти. Данный блок обеспечивает хранение идентификатора (тэга), информационных данных и служебной информации, а также выполняет функции поиска и первичной коммутации данных.

Система хранения данных и признаков реализована на базе синхронных регистров семейства LPM_DFF. Модуль `lpm_dff0` разрядностью 5 бит предназначен для хранения значения тэга. Регистры `lpm_dff1` и `lpm_dff2` обеспечивают хранение двух 8-разрядных слов данных, составляющих одну строку. На триггере `lpm_dff3` реализован бит валидности, указывающий на актуальность хранящихся в строке данных. Все элементы хранения снабжены цепями синхронизации (`clk`), разрешения записи (`write_line`) и асинхронного сброса (`reset`).

Логика идентификации и поиска базируется на цифровом компараторе `lpm_compare0`. Блок выполняет побитовое сравнение текущего тэга адреса (`input_tag`) со значением, хранящимся в регистре тэгов. Результат сравнения поступает на логический элемент AND2 (`inst4`), где объединяется с сигналом бита валидности. Выходной сигнал `line_hit` принимает логическое значение «1» только при одновременном совпадении тэгов и установленном признаке валидности строки.

Узел коммутации выходных данных реализован на мультиплексоре `lpm_mux0`. В зависимости от значения управляющего сигнала смещения (`offset`), мультиплексор транслирует на выходную шину `result[7..0]` либо первое, либо второе слово данных из текущей строки. Данная структура позволяет эффективно выбирать необходимое слово внутри блока при фиксированном попадании в строку.

На рисунке 2.2 представлен полученный блок строки кэш-памяти.

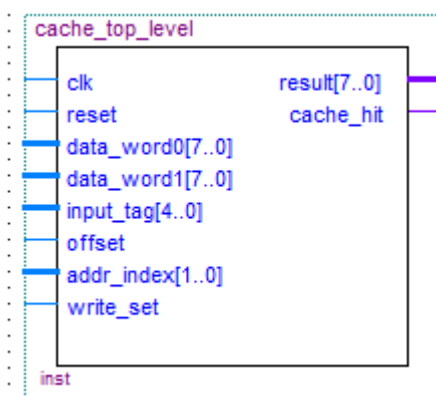


Рисунок 2.2 – Блок строки кэш-памяти

2.3 Моделирование блока строки кэш-памяти

Результаты функционального моделирования блока строки кэш-памяти приведены на рисунке 2.3.

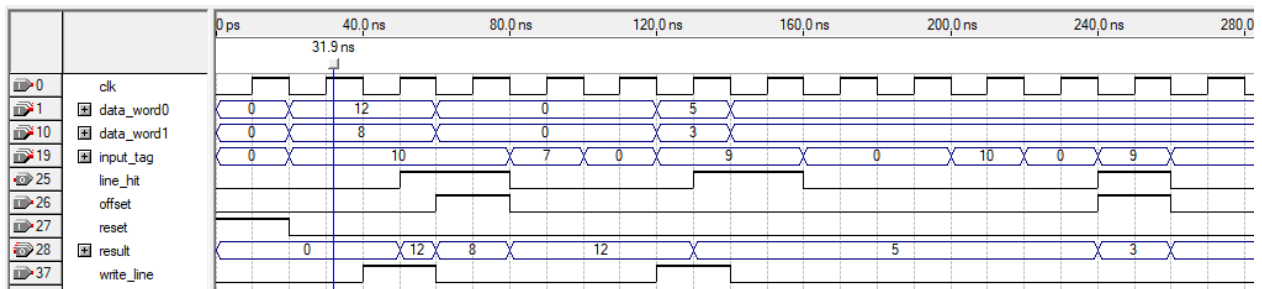


Рисунок 2.3 – Результаты функционального моделирования блока строки кэш-памяти

В начале цикла моделирования (интервал 0–20 нс) подается сигнал асинхронного сброса `reset`, устанавливающий все внутренние регистры в нулевое состояние. На временной отметке 40 нс активируется сигнал разрешения записи `write_line` при значении входного тэга `input_tag` = 10 и данных `data_word0` = 12, `data_word1` = 8. По фронту тактового сигнала `clk` данные успешно фиксируются в соответствующих регистрах хранения.

После завершения цикла записи, при сохранении значения `input_tag` = 10, выходной сигнал попадания `line_hit` переходит в состояние логической «1». Это подтверждает правильность работы цифрового компаратора и блока контроля валидности строки. В момент изменения входного тэга на значение «7» (отметка 80 нс) сигнал `line_hit` мгновенно сбрасывается в «0», что свидетельствует о корректной фиксации «промаха» (Miss).

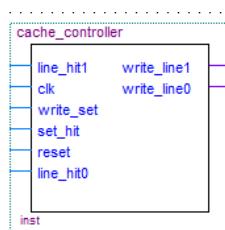
На интервале 60–80 нс продемонстрирована работа мультиплексора слов. При низком уровне сигнала `offset` на выходную шину `result` транслируется значение первого слова строки («12»). Переключение сигнала `offset` в состояние логической «1» приводит к изменению выходных данных на значение второго слова («8»). Аналогичное поведение системы наблюдается на отметке 240 нс для блока данных с тэгом «9».

3.1 Проектирование блока контроллера управления 2-канальным набором кэш-памяти

The screenshot shows a logic design in Quartus II. On the left, there are input buffers for 'line_h0', 'line_h1', 'set_h0', 'set_h1', 'ck', and 'reset'. The 'ck' input is connected to a clock source. The 'reset' input is connected to a constant '1'. The main logic block is a DFF (lpm_dff4) with inputs 'data', 'clock', and 'enable'. The 'data' input is connected to the output of an AND2 gate (inst15). The 'clock' input is connected to the 'ck' input. The 'enable' input is connected to the output of an AND2 gate (inst11). The output of the DFF is connected to a NOT gate (inst11). The output of the NOT gate is connected to an AND2 gate (inst24). The output of the AND2 gate (inst24) is connected to an OR2 gate (inst25). The output of the OR2 gate (inst25) is connected to an AND2 gate (inst26). The output of the AND2 gate (inst26) is connected to a 4-bit bus 'write_line0'. There are also other logic blocks like AND2 (inst12, inst13, inst14, inst16, inst17, inst18, inst19, inst20, inst21, inst22, inst23, inst24, inst25, inst26), OR2 (inst22, inst23, inst24, inst25, inst26), AND3 (inst23), OR3 (inst24), AND4 (inst24), OR4 (inst25), AND5 (inst25), and OR5 (inst26).

Данный функциональный блок является управляющим узлом кэш-памяти. В его основе лежит регистр состояния очереди `lpm_dff4`, функционирующий как Т-триггер. Логическая комбинация элементов AND2 (`inst15`) и NOT (`inst12`) обеспечивает переключение очереди замещения исключительно в моменты возникновения промахов (`set_hit = 0`) при активном сигнале записи.

На рисунке 3.2 представлен полученный блок контроллера управления 2-канальным набором кэш-памяти.



8

3.2 Моделирование блока контроллера управления 2-канальным набором кэш-памяти

Результаты функционального моделирования блока контроллера управления 2-канальным набором кэш-памяти приведены на рисунке 3.3.

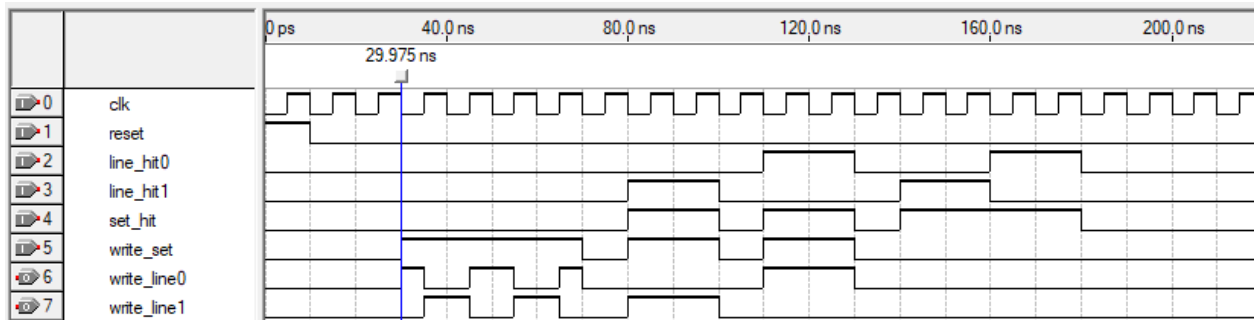


Рисунок 3.3 – Результаты функционального моделирования блока контроллера управления 2-канальным набором кэш-памяти

В интервале моделирования 40–80 нс продемонстрирована работа алгоритма при возникновении «промахов» ($set_hit = 0$). При подаче последовательных импульсов записи $write_set$, контроллер поочередно активирует выходные линии $write_line0$ и $write_line1$. Это доказывает правильное функционирование внутреннего триггера очереди, который переключает «путь» записи после каждого заполнения новой строки.

На временной отметке 120 нс имитируется ситуация попадания в первый канал ($line_hit0 = 1$). При подаче сигнала записи $write_set$ контроллер активирует строго соответствующую линию $write_line0$. Важно отметить, что на отметке 140 нс при попадании во второй канал ($line_hit1 = 1$) сигнал записи транслируется на $write_line1$. Это подтверждает приоритет сигналов попадания над состоянием триггера очереди, что исключает дублирование тегов в разных путях одного набора.

Моделирование показало, что при возникновении попадания ($set_hit = 1$) состояние указателя очереди остается неизменным. Это гарантирует сохранение порядка вытеснения «наиболее давнего хранения», так как обновление существующей информации не считается загрузкой нового блока данных.

4 Блок 2-канального ассоциативного набора кэш-памяти

4.1 Проектирование блока 2-канального ассоциативного набора кэш-памяти

Структурная схема блока 2-канального ассоциативного набора кэш-памяти представлена на рисунке 4.1.

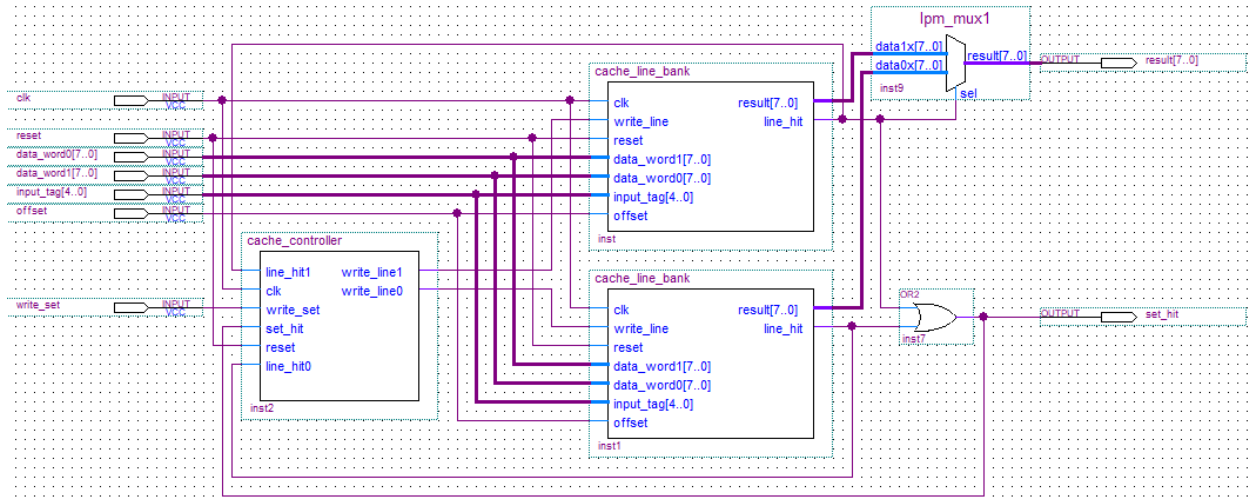


Рисунок 4.1 – Структурная схема блока 2-канального ассоциативного набора кэш-памяти

Представленная схема объединяет в себе ранее разработанные блоки в единую функциональную структуру — двухканальный набор (Set). Данный модуль реализует логику k-мерного частичного ассоциативного отображения для $k=2$.

В состав схемы включены два идентичных экземпляра базового модуля строки (**cache_line_bank**), спроектированного на предыдущем этапе. Эти блоки обеспечивают параллельное хранение тегов и данных, а также независимую проверку на наличие «попадания» в каждом из двух каналов (путей) кэша.

Управление совместной работой каналов осуществляется с помощью специализированного контроллера (**cache_controller**), также разработанного ранее. Контроллер анализирует состояние сигналов попадания от обоих банков и, согласно алгоритму замещения, распределяет внешние команды записи. Благодаря интеграции контроллера и банков в один символ, обеспечивается автоматическое обновление данных при попадании и корректное вытеснение строк при промахмах.

Для формирования единого выхода набора применены дополнительные комбинационные схемы: элемент **OR2** объединяет локальные признаки попадания в общий сигнал **set_hit**, а мультиплексор **lpm_mux1** осуществляет селекцию информационных шин.

На рисунке 4.2 представлен полученный блок 2-канального ассоциативного набора кэш-памяти.

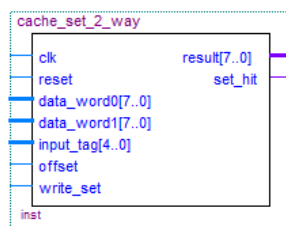


Рисунок 4.2 – Блок 2-канального ассоциативного набора кэш-памяти

4.2 Моделирование блока 2-канального ассоциативного набора кэш-памяти

Результаты функционального моделирования блока 2-канального ассоциативного набора кэш-памяти приведены на рисунке 4.3.

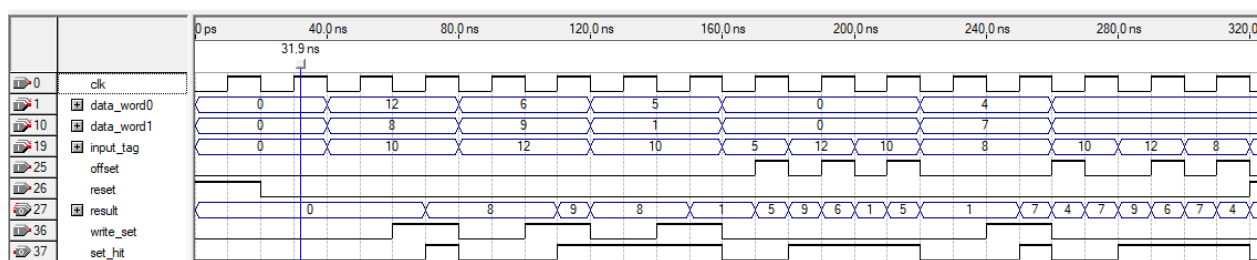


Рисунок 4.3 – Результаты функционального моделирования блока 2-канального ассоциативного набора кэш-памяти

На интервале 120–140 нс зафиксирована повторная запись по тегу «10». Благодаря разработанной логике контроллера, система идентифицирует «попадание» ($set_hit = 1$) и производит перезапись данных (значения «5» и «1») в соответствующую строку, не изменяя при этом состояние очереди замещения.

На временном отрезке 160–200 нс показан процесс вытеснения данных при заполнении набора. Последовательная запись тега «5» приводит к автоматическому удалению одного из ранее хранимых блоков, так как емкость набора (2 пути) исчерпана. Последующее обращение к вытесненному тегу «10» (отметка 200 нс) ожидаемо фиксирует промах ($set_hit = 0$), что подтверждает работоспособность алгоритма наиболее древнего хранения.

Диаграмма наглядно иллюстрирует процесс коммутации данных на шину *result*. Выходной мультиплексор набора корректно транслирует информацию из того канала, который выдал сигнал совпадения тега. Дополнительно подтверждена работа сигнала *offset*, который позволяет выбирать конкретное слово (например, значения «4» и «7» на отметке 280 нс) внутри уже найденной строки кэша.

5 Схема кэш-памяти

5.1 Проектирование схемы кэш-памяти

Структурная схема кэш-памяти представлена на рисунке 5.1.

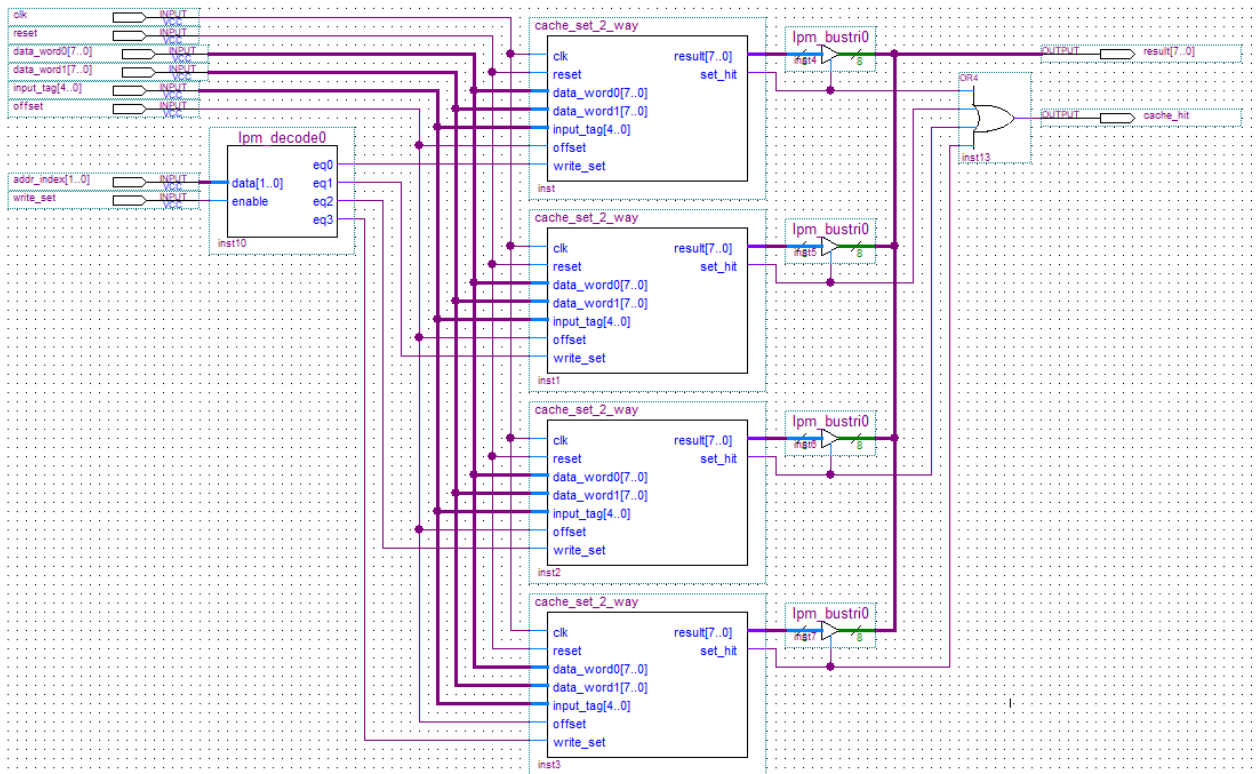


Рисунок 5.1 – Структурная схема кэш-памяти

Представленная схема является финальным этапом проектирования и представляет собой полную архитектуру 2-канальной множественно-ассоциативной кэш-памяти объемом 8 строк. Система построена по иерархическому принципу с использованием четырех идентичных наборов, каждый из которых содержит по два пути хранения.

Узел дешифрации и выбора набора реализован на базе модуля `lpm_decode0`. На вход дешифратора поступают биты адреса `addr_index[1..0]`, которые соответствуют полю Set (Индекс) адресного слова. Дешифратор преобразует 2-битный индекс в один из четырех сигналов разрешения записи (`eq0...eq3`). Это гарантирует, что при обращении процессора к памяти активируется строго определенный набор, соответствующий заданному адресу, что исключает некорректную запись данных.

Коммутация выходных данных организована с помощью буферов с тремя состояниями (`lpm_bustri0`). Использование Z-состояния позволяет объединить выходы всех четырех наборов в единую 8-разрядную шину `result[7..0]`. Управление буферами осуществляется сигналами попадания

(set_hit) от соответствующих наборов: данные выдаются на общую шину только тем блоком, в котором произошло совпадение тегов.

Формирование результирующих признаков завершается на элементе логического сложения OR4 (inst13). Данный элемент объединяет локальные сигналы попадания от каждого из четырех наборов в единый глобальный сигнал попадания кэш-памяти — cache_hit. Логическая «1» на данном выходе информирует внешнюю систему о том, что запрашиваемые данные найдены в кэше и транслируются на выходную шину, в то время как логический «0» сигнализирует о необходимости обращения к основной памяти (ОЗУ).

5.2 Моделирование схемы кэш-памяти

Результаты функционального моделирования блока 2-канального ассоциативного набора кэш-памяти приведены в приложении А.

На представленной временной диаграмме отражены результаты комплексного тестирования всей архитектуры кэш-памяти. На графике зафиксированы процессы последовательной записи данных во все четыре набора, работа дешифратора индексов и функционирование выходных Z-буферов. Итоговая симуляция подтверждает, что система корректно управляет всеми восемью строками хранения.

В первой фазе моделирования (0–160 нс) продемонстрирована работа дешифратора адреса. При изменении сигнала addr_index от 0 до 3, импульсы записи write_set распределяются строго по соответствующим наборам. В нижней части диаграммы (внутренние регистры) отчетливо видно, что данные фиксируются только в том наборе, чей индекс указан в адресной шине, при этом содержимое остальных наборов остается неизменным.

Во второй фазе (160–320 нс) проверяется логика ассоциативности и выходной коммутации. При подаче на вход ранее записанных тегов и соответствующих им индексов, система мгновенно формирует глобальный сигнал попадания cache_hit. Одновременно с этим активируются соответствующие выходные буферы, и на общую шину result транслируются данные из выбранного набора. Отсутствие неопределенных состояний (X) и конфликтов на шине подтверждает правильность объединения блоков через Z-состояние.

В заключительной части симуляции подтверждена работа алгоритма замещения в масштабе всей системы. При повторном обращении к набору с индексом «0» и записи нового тега, происходит вытеснение самой старой строки в данном наборе, в то время как данные в других наборах (с индексами 1, 2 и 3) сохраняются. Это доказывает полную функциональную пригодность спроектированной 2-канальной множественно-ассоциативной кэш-памяти.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была спроектирована и успешно протестирована модель 2-канальной ассоциативной кэш-памяти объемом 8 строк. Весь проект был реализован по иерархическому принципу: от разработки отдельной строки и контроллера управления до сборки финальной схемы из четырех наборов. Такой подход позволил создать логичную и удобную в отладке систему, где каждый блок отвечает за свою задачу.

Особое внимание в работе было уделено логике управления замещением. Разработанный контроллер на базе Т-триггера реализует алгоритм «наиболее давнего хранения» и корректно обрабатывает ситуацию обновления данных при попадании. Это исключает дублирование одинаковых тегов в разных путях и оптимизирует использование памяти.

На этапе проектирования были проведены расчеты адресного пространства. С учетом заданной кратности ОЗУ была определена 8-битная структура адреса, включающая 5-битный тэг, 2-битный индекс набора и 1-битное смещение. Это обеспечило точную идентификацию данных и правильную работу дешифратора при выборе одного из четырех наборов кэша.

Результаты функционального моделирования в среде Quartus II полностью подтвердили работоспособность всех узлов системы. Временные диаграммы наглядно показали, что кэш правильно фиксирует попадания и промахи, корректно коммутирует данные через Z-буферы и вовремя вытесняет старые строки при заполнении памяти. Спроектированное устройство является завершенным и соответствует всем техническим требованиям.

Приложение А (обязательное)

Функциональное моделирование кэш-памяти

